

Gruppo 24-14, Parte 2 Completa

Paolo Casagrande e Tufa Fabiano

Sommario

Questa relazione spiega come abbiamo svolto il testo del progetto di un linguaggio sintassi ispirata a Chapel, scoping statico e tipi espliciti. La pipeline comprende lexer (Alex), parser (Happy), type checker che riporta tutti gli errori trovati, generatore TAC e pretty-printer del sorgente e del codice intermedio, dove ogni identificatore è annotato con la riga di dichiarazione (`nome_riga`). L'ambiente è passato esplicitamente e la generazione usa la state monad. Abbiamo anche predisposto un makefile che costruisce l'eseguibile e make demo lo esegue su una cartella di test.

Indice

1	Architettura generale e grammatiche libere dal contesto	2
2	Grammatica e Sintassi	3
3	Lexer	4
4	Parser	4
5	Scelte tecniche implementative	5
6	Implementazione del type checker	7
6.1	TypeErrors.hs	7
6.2	SemTypes.hs	7
6.3	Environment.hs	7
6.4	TypedAst.hs	8
6.5	TypeCheck.hs	8
7	Progettazione del TAC	10
8	Generazione del TAC	11
9	Pretty-printer del TAC e Main	12

1 Architettura generale e grammatiche libere dal contesto

Versioni degli strumenti utilizzati:

- GHC: 9.2.8
- BNFC: 2.9.1
- Alex: 3.5.4.2
- Happy: 2.2

Per l'architettura del progetto abbiamo deciso di adottare un approccio a moduli separati, come consigliato nel corso, restando in linea con le specifiche della traccia. Più precisamente, abbiamo suddiviso i moduli di lavoro in:

- **Definizione della grammatica**
- **Lexer**
- **Parser che genera un AST non attribuito**
- **Type checker**
- **Generatore TAC**
- **Pretty printer**

2 Grammatica e Sintassi

Come si può leggere nel file `Linguaggio.cf`, la grammatica prevede variabili globali, procedure/funzioni, parametri, tipi, blocchi di codice, istruzioni, controlli, operatori logici, operatori di confronto, operatori aritmetici, operatori unari, indice array e chiamata di funzione, espressioni, gestione parentesi e gestione dei commenti. Le scelte tecniche che abbiamo preso per la grammatica e la sintassi sono:

- Abbiamo seguito lo schema "**categorie numerate + coercions**" per la precedenza degli operatori, in modo da ridurre la probabilità di conflitti ed evitare eventuali ambiguità nei passaggi successivi.
- Come separatore abbiamo scelto di utilizzare il punto e virgola `;` come in Chapel
- Per gli array abbiamo scelto una sintassi simile a Chapel composta così: `[lo..hi] Type` (ovvero `[min..max] Tipo`); per costruire un array abbiamo introdotto anche i letterali array, con la sintassi `[e1, ..., en]`
- Per i puntatori abbiamo scelto la seguente sintassi: `c_ptr(Type)/c_ptrTo(...)/*`; `c_ptr` è un nome preso della libreria di interoperabilità C di Chapel
- I parametri possono essere passati per riferimento o per valore (default)
- Sia le funzioni che le procedure devono tornare un valore obbligatoriamente, che può essere `void`
- Abbiamo deciso di rendere possibile la dichiarazione di variabili e funzioni dentro qualunque blocco; inoltre si possono anche inserire blocchi dentro ad altri blocchi, ricorsivamente
- Abbiamo aggiunto i commenti a riga singola e multi-riga
- Abbiamo aggiunto cicli indeterminati e determinati di iterazione: insieme al `while cond {...}` abbiamo implementato anche il `do {...} while cond`; con controllo in coda, e abbiamo aggiunto il `for i in a..b {...}` per l'iterazione su un intervallo con la variabile di ciclo che è sempre di tipo intero
- Abbiamo aggiunto le istruzioni `break`; e `continue`;
- Abbiamo aggiunto gli assegnamenti composti `+=`, `-=` e `*=`
- Abbiamo introdotto l'if-then-else anche come espressione, con la sintassi `if cond then e1 else e2`, dove entrambi i rami sono espressioni

Per evitare ambiguità e dangling-else abbiamo voluto inserire tra parentesi grafe il corpo di tutti i costrutti di controllo e di iterazione. Infatti, eseguendo il comando per la creazione della grammatica con BNFC, Happy non ha segnalato nessun conflitto shift-reduce o reduce-reduce su 78 regole.

3 Lexer

Abbiamo verificato le scelte prese nella grammatica attraverso dei test (visibili nella cartella `tests/lexer`). Li abbiamo eseguiti sul lexer per controllare se fosse necessario aggiustare qualche comportamento:

- Le regex per i commenti scartano fino alla fine della riga per i commenti a riga singola `//` e per quelli multi riga `/* */` vengono scartate tutte le righe fino alla sequenza asterisco-slash attaccati; i test ci hanno confermato che funzionano
- La regex copre la notazione scientifica con `3.14` e anche `1.5e10/1.5e-3`; non è coperto il caso con la `E` maiuscola né il segno `+` davanti all'esponente (tipo `1.5e+10`). Abbiamo scelto di tenere la versione più semplice, che soddisfa comunque il requisito di poter scrivere numeri in notazione scientifica.
- Il funzionamento dei caratteri viene riconosciuto correttamente: `\", \\, \', \n, \t, \r, \f` mentre per i caratteri `\` e `n` abbiamo usato una funzione `unescapeInitTail`, con il test che dava esito positivo
- Il token `Ident` accetta lettere maiuscole, minuscole, cifre e underscore dopo il primo carattere, permettendo camelCase, UPPER_CASE e nomi misti superando i test

Una volta che abbiamo eseguito i test e che sono stati superati con esito positivo siamo passati alla fase successiva, quella di controllo del parser.

4 Parser

Anche per il parser abbiamo voluto controllare attraverso dei test se fosse tutto corretto o se fossero necessarie delle modifiche. Abbiamo eseguito due tipi di test controllando sia il riconoscimento degli errori che l'analisi sintattica.

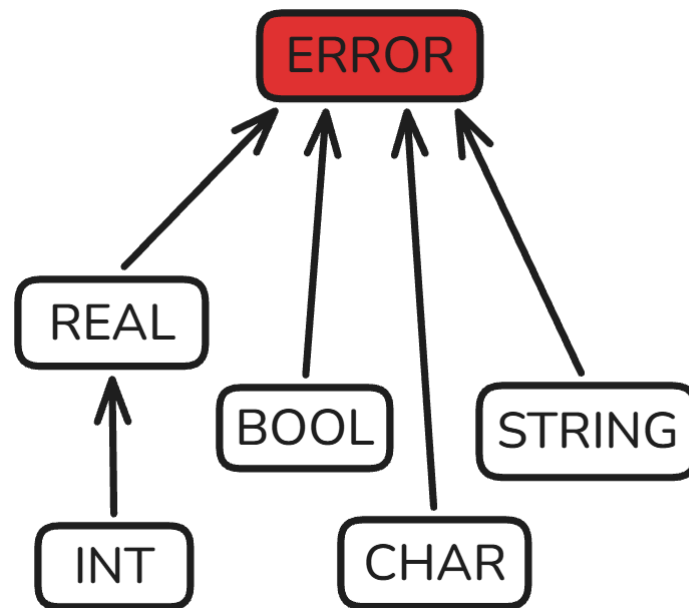
Per gli errori abbiamo eseguito i test contenuti nella cartella `tests/errors`, con i quali abbiamo provato a far fallire il parsing usando input non validi e in tutti e i casi il messaggio d'errore era chiaro e puntava al punto giusto. Abbiamo scelto di lasciare il messaggio di default nel caso di un errore di sintassi per concentrarci maggiormente sui messaggi di errore che riguardano il type checker richiesto dalla traccia che implementeremo in seguito.

Per l'analisi sintattica, invece, abbiamo eseguito i test contenuti nella cartella `tests/errors`, dove abbiamo verificato che per input corretti il parser producesse l'albero giusto seguendo le regole grammaticali del linguaggio.

5 Scelte tecniche implementative

Prima di scrivere il type checker abbiamo preso alcune decisioni che abbiamo scelto di esporre: la visibilità delle variabili, la compatibilità fra i tipi e la validità delle funzioni nell'ambiente.

Per la visibilità delle variabili abbiamo scelto la modalità "dal punto di dichiarazione alla fine". Ci sembrava la soluzione più semplice per soddisfare la richiesta del testo di utilizzare lo scoping statico e per rimanere in linea con Chapel. Questa modalità però ci obbliga a tenere d'occhio il caso di funzioni con mutua ricorsione; abbiamo quindi adottato un meccanismo di pre-scansione che controllasse le intestazioni prima dei corpi e che le tenesse in memoria (la funzione `preScanFunctions`, spiegata nel capitolo successivo).



Come tipi ammessi dal linguaggio abbiamo scelto Interi, Booleani, Caratteri, Stringhe e Reali. La figura sopra mostra le relazioni di compatibilità tra i vari tipi, dove il tipo `int` è compatibile con `real` mentre viceversa il tipo `real` (avendo una parte decimale) non è compatibile con `int`. Per gli array e i puntatori abbiamo deciso che un array è compatibile solo con un array del tutto identico (quindi con gli stessi limiti e lo stesso tipo di elementi) e la stessa regola vale anche per i puntatori.

Per l'uguaglianza (`==`, `!=`) e per gli operatori d'ordine (`<`, `<=`, `>`, `>=`) abbiamo visto che calcolare il sup dei due tipi e accettare l'operazione se il risultato non dava errore non bastava. Anche perchè ha sempre successo sia con l'uguaglianza tra due tipi uguali (es. `string==string`) che con gli operatori di ordine (es. `char<char`) anche se non avrebbe senso. Abbiamo quindi aggiunto un controllo al calcolo del sup che controllasse l'appartenenza del tipo a due categorie:

- L'uguaglianza è ammessa solo fra booleani, caratteri, interi e reali
- Gli operatori d'ordine sono ammessi solo fra interi e reali

In entrambi i casi il sup viene calcolato per calcolare il cast tra intero e reale ma non basta per far accettare l'operazione.

Per la ridichiarazione e lo shadowing abbiamo scelto una regola unica per tutto il linguaggio. Siccome le variabili e le funzioni occupano due spazi di nomi separati, una variabile e una funzione con lo stesso nome non vanno in conflitto. Dichiarare due volte lo stesso nome all'interno dello stesso blocco (e quindi nello stesso spazio di nomi) da errore. Dichiararlo però in un blocco più

interno è ammesso, quindi si possono ridefinire localmente funzioni o variabili. Anche i parametri di funzione generano un errore di ridichiarazione.

Il `for` dichiara la propria variabile di ciclo sempre di tipo intero. Questa variabile può essere letta all'interno del blocco ma non modificata; anche gli estremi dell'intervallo devono essere di tipo intero.

Per il costrutto condizionale usato come espressione (`if cond then e1 else e2`) abbiamo impostato che la condizione sia di tipo booleano e che i due rami abbiano un tipo in comune come per la relazione di sup per gli operatori binari, inserendo un cast esplicito sul ramo se il tipo non coincide con quello risultante. Il tipo del risultato non può essere un array, infatti il valore di un'espressione deve stare in un unico temporaneo, che nel nostro TAC può contenere solo un tipo primitivo, e questo vincolo esclude un array come tipo di ritorno di una funzione. I puntatori ad array però sono ammessi, perché un puntatore è comunque un valore singolo.

Per gli assegnamenti composti (`+=`, `-=`, `*=`) abbiamo scelto che il lato sinistro fosse una l-expression modificabile e che entrambi gli operandi siano di tipo aritmetico. Il tipo del risultato dell'operazione è calcolato come sup dei due tipi e deve coincidere con il tipo del lato sinistro.

Infine abbiamo deciso che ogni array deve essere obbligatoriamente inizializzato al momento della dichiarazione tramite un letterale array. Un letterale è tipizzato con il tipo dell'array atteso: deve infatti avere lo stesso numero di elementi previsto dall'intervallo dichiarato e ogni elemento deve avere un tipo compatibile con quello degli elementi dell'array. Se il tipo degli elementi è anche quello un array, ogni elemento del letterale deve essere a sua volta un letterale che rispetti la stessa regola.

6 Implementazione del type checker

Per l'implementazione del type checker abbiamo scritto cinque file, che andremo a spiegare sotto:

- `TypeErrors.hs`
- `SemTypes.hs`
- `Environment.hs`
- `TypedAst.hs`
- `TypeCheck.hs`

6.1 `TypeErrors.hs`

Per rappresentare gli errori abbiamo definito un nuovo tipo, `TypeError`, che comprende la posizione e il messaggio di errore. In questo modo riusciamo a riportare tutti gli errori senza fermarci al primo incontrato. Abbiamo usato il `Maybe` perché non è detto che un errore abbia sempre una posizione all'interno del codice. Abbiamo inoltre aggiunto la funzione `showType`, che converte un tipo nella sua rappresentazione testuale: questo ci permette di includere nel messaggio i tipi coinvolti rendendo l'errore più preciso e più facile da individuare.

6.2 `SemTypes.hs`

Per gestire i calcoli e includere anche il tipo errore appena introdotto abbiamo adottato un nuovo modo per rappresentare i tipi, contenuto in questo file. I tipi semplici vengono convertiti restituendo il tipo equivalente, mentre per gli array e puntatori la funzione richiama se stessa sul tipo annidato.

Questo file contiene anche le funzioni per gestire la compatibilità dei tipi, basate sulla funzione `sup`, che calcola il tipo "superiore" fra due tipi dati. Per l'uguaglianza e per gli operatori d'ordine, però, il calcolo del `sup` non basta: come spiegato nel capitolo precedente. Abbiamo quindi aggiunto due funzioni dedicate, `eqOp` e `ordOp`: entrambe calcolano comunque il `sup` dei due tipi (per il cast implicito) ma controllano anche che il tipo risultante appartenga all'insieme ammesso da quell'operatore, rifiutando l'operazione altrimenti.

Abbiamo inoltre esteso la funzione `mathType` alle operazioni aritmetiche binarie: un'unica funzione stabilisce se un tipo è aritmetico sia dal meno unario sia da `+`, `-`, `*` e `/`. L'ultima funzione del file controlla se un tipo è assegnabile ad un determinato target.

6.3 `Environment.hs`

Questo file contiene l'ambiente, che tiene in memoria due mappe separate, una per le variabili e una per le funzioni: un nome usato da solo lo si cerca fra le variabili, un nome seguito da parentesi fra le funzioni. Per ogni variabile teniamo il suo tipo e la posizione, mentre per ogni funzione teniamo la lista dei parametri (con la loro modalità) e il loro tipo di ritorno. Il corpo delle funzioni viene letto solo quando serve.

Abbiamo aggiunto un flag che indica se nel punto in cui ci troviamo siamo dentro il corpo di un ciclo: questo ci serve se un `break` o un `continue` compaiono in una posizione corretta. Questo flag viene azzerato quando si entra nel corpo di una funzione dichiarata all'interno di un ciclo, in modo che le due istruzioni non possano sovrapporsi per errore.

Per ogni variabile memorizziamo anche se la variabile è modificabile (falso solo per la variabile introdotta in un `for`) e la modalità con cui un nome stato passato (per valore o per riferimento). Quest'ultima informazione viene propagata fino all'albero tipizzato perché il generatore di TAC ne tiene conto per generare l'accesso ad un parametro passato per riferimento.

6.4 TypedAst.hs

Questo file contiene un albero simile a quello del parser, però ogni nodo contiene anche il proprio tipo e la posizione dove è dichiarato. Abbiamo anche incluso un tipo associato al casting, che memorizza la conversione invece di eseguirla in modo che venga fatta solo nel codice finale. Le istruzioni eseguono azioni senza produrre valori, quindi i loro nodi nell'AST non hanno un tipo (a differenza delle espressioni). La struttura dati del parametro invece include esplicitamente sia il tipo che il nome dell'identificatore dentro una lista.

6.5 TypeCheck.hs

Questo file contiene il type checker, dove per ogni elemento abbiamo scelto un comportamento. Siccome il tipo dei letterali è noto, questi non consultano l'ambiente. Per le espressioni, se la ricerca ha successo si costruisce il nodo col tipo e la posizione di dichiarazione; se fallisce, si costruisce un nodo con tipo errore e senza posizione e si segnala l'errore; l'analisi poi può proseguire invece di fermarsi al primo errore.

Per le operazioni aritmetiche, si controllano prima i due operandi separatamente, verificando che siano di tipo aritmetico, e poi si calcola il tipo "superiore". Se il risultato non è errore va tutto bene; se è errore ma uno dei due operandi lo era già, non si aggiunge un secondo messaggio, per evitare di generare errori derivati dallo stesso problema. Se i tipi sono incompatibili si genera un errore nuovo. Il tipo di casting viene inserito solo quando il tipo dell'operando non coincide col tipo risultante.

Per le operazioni di uguaglianza e di ordine seguiamo lo stesso schema delle operazioni aritmetiche, ma usando rispettivamente `eqOp` e `ordOp` al posto del semplice `sup`, in modo da rifiutare anche i confronti sintatticamente corretti ma non ammessi dal linguaggio, come un'uguaglianza fra due array o un ordinamento fra due caratteri.

Per array e puntatori servono controlli specifici per verificare che quella cosa abbia davvero un punto in memoria a cui puntare. Per questo abbiamo raccolto la logica in un'unica funzione per il controllo di "l-expression modificabile", usata dall'assegnamento semplice, dall'assegnamento composto, dal passaggio di un argomento per riferimento e dall'operatore che prende l'indirizzo di una variabile.

Per le chiamate di funzione facciamo una ricerca del nome nella memoria pre-scansionata. Se la funzione esiste, gli argomenti sono controllati uno alla volta e confrontati con i parametri, dove i tipi vengono confrontati. Se i parametri sono passati per riferimento, serve il tipo esattamente identico e l'argomento deve avere già un posto in memoria, verificato con la funzione descritta sopra.

Per l'assegnamento, si controlla che il lato sinistro abbia un luogo in memoria e che il tipo del lato destro sia compatibile con quello del sinistro. Per l'assegnamento composto (`+=`, `-=`, `*=`) verifichiamo in più che il lato sinistro sia modificabile, che entrambi gli operandi siano di tipo aritmetico e che il tipo del risultato dell'operazione coincida esattamente con quello del lato sinistro.

Per `if/while`, si controlla che la condizione sia di tipo booleano e poi si controllano i corpi. Stessa cosa vale per il `do-while`. Per l'`if-then-else` usato come espressione controlliamo che la condizione sia booleana e calcoliamo il sup dei tipi dei due rami, rifiutando il caso in cui il tipo risultante sia un array. Per un `return`, il tipo del valore restituito va confrontato con quello atteso dalla funzione, che è tenuto in memoria; un controllo sul tipo di ritorno ci permette di rifiutare di ritornare un array.

Per la dichiarazione di un array controlliamo che l'inizializzatore sia sempre presente e che sia un letterale array: viene inoltre confrontato il numero di elementi con la dimensione dichiarata, verificata la compatibilità del tipo di ciascun elemento (inserendo eventuali cast dove serve) e richiamandosi ricorsivamente su ogni elemento annidato se gli elementi sono a loro volta array.

Abbiamo anche incluso una parte per la doppia passata della mutua ricorsione, dove scandiamo tutte le istruzioni memorizzando solo le dichiarazioni delle funzioni, ignorando il resto senza leggerne il corpo. Così riusciamo a rendere una funzione dichiarata in un punto inferiore chiamabile da una in un punto superiore. La pre-scansione confronta i nomi già incontrati soltanto all'interno dello stesso blocco che si sta scandendo, e non anche con eventuali indentazioni. Con lo stesso meccanismo viene rilevata anche la ridichiarazione delle variabili: in ogni blocco viene tenuta traccia dei nomi già dichiarati (separatamente rispetto alle funzioni), e dichiarare di nuovo lo stesso nome nello stesso spazio genera un errore, mentre uno shadowing in un blocco annidato si può fare.

Per una dichiarazione di variabile con valore iniziale viene memorizzato il tipo dichiarato, non quello del valore iniziale, anche in caso di errore: abbiamo deciso che il tipo dichiarato rimane valido per il resto del programma. Per una dichiarazione di funzione si controlla il corpo e se la funzione non era già stata registrata dalla pre-scansione la si memorizza anche nell'ambiente esterno.

Abbiamo testato il TypeChecker con i file contenuti nella cartella `tests/typecheck` e `tests/errors`, eseguendo sette test diversi:

- Eseguendo un file contenente cinque errori diversi, verificando che venissero riportati tutti e cinque insieme
- Chiamando due funzioni annidate che si richiamano a vicenda, per verificare la mutua ricorsione
- Provando a passare un numero per riferimento e mescolando puntatori di tipo diverso, verificando che entrambi i casi venissero rifiutati
- Confrontando due array con `==`, verificando che venisse rifiutato
- Confrontando due caratteri con un operatore d'ordine, verificando che venissero rifiutati
- Usando `break` al di fuori del corpo di un ciclo, verificando che venisse rifiutato
- Modificando all'interno del corpo di un `for` la sua variabile di ciclo, verificando che fosse rifiutato

7 Progettazione del TAC

In questa parte abbiamo ragionato su come implementare il three-address code come richiesto dal testo.

Per gli indirizzi abbiamo previsto quattro categorie, basandoci sui modi in cui un valore può comparire in un'istruzione: un nome di variabile, un valore letterale, un valore temporaneo, e un'area usata per le stringhe letterali (queste variabili contengono l'indirizzo del contenuto). Ogni indirizzo contiene il suo tipo che serve sia per scegliere l'istruzione giusta in fase di generazione, sia più avanti nel progetto per annotare le variabili con la riga di dichiarazione in fase di stampa. Per le istruzioni abbiamo previsto un costruttore per ogni operazione del linguaggio; in questo modo le operazioni aritmetiche, di confronto e logiche hanno un costruttore specifico per ogni combinazione di operazione e tipo (ad esempio la somma fra interi e la somma fra reali sono due istruzioni diverse). In questo modo non dobbiamo controllare il tipo dei dati. Le altre categorie di istruzioni comprese sono:

- Copie semplici
- Salti
- Lettura e scrittura in memoria
- Chiamate di funzione

Abbiamo gestito anche il caso per il casting associandolo direttamente al nodo corretto dell'AST tipizzato. Per la sequenza di codice abbiamo scelto di rappresentare le etichette come elemento a sé stante della lista, non "attaccate" all'istruzione che segue. Così facendo possiamo rappresentare correttamente anche un blocco vuoto senza perdere l'etichetta che deve comunque comparire nel codice finale. Infine, abbiamo deciso che gli accessi ad array multidimensionali fossero calcolati ricorsivamente e che le guardie booleane di if e while adottassero il jumping code, saltando direttamente alle etichette senza generare valori booleani intermedi, mantenendo il short-circuit.

8 Generazione del TAC

In questa fase abbiamo lavorato sulla generazione del TAC. Come vincoli del testo dobbiamo valutare l-value prima di r-value in un assegnamento e lo short-circuit obbligatorio nelle condizioni di `if/while`. Per generare nuovi temporanei ed etichette senza ripetere i nomi, e senza usare variabili modificabili sul posto, abbiamo deciso di usare la state monad, con due contatori separati, uno per i temporanei e uno per le etichette. Come ordine di valutazione abbiamo scelto da sinistra a destra, siccome corrisponde all'ordine in cui l'espressione è scritta nel programma. Per calcolare l'indirizzo di una variabile, di un elemento di array o del contenuto puntato da un puntatore, abbiamo usato una funzione che distingue i tre casi, usata sia quando serve l'indirizzo per l'indicizzazione sia quando serve come l-value di un assegnamento. Per le condizioni booleane composte da `&& / ||` abbiamo applicato lo schema del jumping code, così quando il risultato è già determinato dal primo operando il codice del secondo non viene generato, in modo da realizzare lo short-circuit richiesto dal testo. Per un confronto o un'espressione booleana semplice non composta, si genera invece il valore e un salto condizionato esplicito. Questo schema è stato esteso anche ai casi in cui un'espressione booleana composta compare come valore e non come guardia. Anche in questi casi riusciamo il jumping invece di generare un'istruzione binaria generica che valuterebbe entrambi gli operandi. Inoltre, per l'assegnamento, la valutazione di l-value prima di r-value richiesto dal testo si può vedere nel caso di scrittura su un elemento di array o attraverso un puntatore, dove si calcola prima l'indirizzo della parte sinistra e poi si genera il codice del valore di destra. Per una variabile semplice l'indirizzo è immediato.

Per il ciclo `while`, abbiamo pensato di saltare subito alla condizione, eseguire il corpo, e poi valutare la condizione, tornando al corpo se è vera o uscendo se è falsa. Così facendo evitiamo di controllare la condizione due volte a ogni giro. Lo stesso schema vale per il `do-while`: si esegue il corpo, si valuta la condizione, e si torna al corpo se è vera. Per il `for` invece generiamo l'inizializzazione della variabile di ciclo con l'estremo inferiore, un confronto con l'estremo superiore prima di ogni giro, il corpo, e infine l'incremento della variabile e il salto indietro al confronto. Le istruzioni `break` e `continue` sono tradotte come salti diretti rispettivamente all'etichetta di uscita e all'etichetta di ripetizione del ciclo in cui compaiono. Per l'assegnamento composto (`op=`) generiamo prima l'indirizzo del lato sinistro, leggiamo il valore da quell'indirizzo, generiamo il valore del lato destro, calcoliamo il risultato dell'operazione e scriviamo il risultato allo stesso indirizzo calcolato all'inizio senza calcolarlo due volte. Per l'`if-then-else` usato come espressione riutilizziamo lo schema di salti dell'`if` però entrambi i rami scrivono il proprio risultato nello stesso temporaneo che viene poi usato per il valore finale dell'espressione.

Per le chiamate di funzione abbiamo due casi per i parametri scalari: per valore si genera il valore dell'espressione con una copia, modificabile dalla funzione chiamata senza effetti esterni, mentre per riferimento si genera invece il suo indirizzo.

Per un array passato per valore il nostro TAC non copia l'intero blocco di memoria in un passaggio solo: viene copiato un elemento alla volta dall'array in una nuova variabile. In questo modo l'array copiato non condivide memoria con quello originale.

Ogni funzione incontrata durante la generazione produce il proprio codice separato dal resto, e non "in mezzo" al codice di chi la contiene. Infine, in fondo al corpo di ogni funzione che restituisce `void` (una procedura) generiamo sempre un'istruzione di `return` finale, anche se il programma non la scrive esplicitamente come ultima istruzione: se il corpo termina già con un `return`, quello aggiunto da noi è visto come codice irraggiungibile e innocuo, altrimenti diventa l'uscita effettiva della procedura.

Abbiamo eseguito dei test con diversi programmi di prova: uno con operazioni aritmetiche e un `if`, uno con array, un `while`, puntatori e una chiamata per riferimento, indicizzazione tramite puntatore, copia di array passati per valore, assegnamenti composti, short-circuit fuori dalle guardie. In tutti i casi l'ordine delle istruzioni generate rispettava esattamente l'ordine l-value prima di r-value richiesto.

9 Pretty-printer del TAC e Main

L'ultima parte di questa relazione parla di come abbiamo scritto il pretty-printer e il main. Nel pretty printer abbiamo deciso di indicare un identificatore che viene da una variabile del programma insieme alla riga in cui è stata dichiarata, attaccata al nome, ad esempio `t37 = x_12 + 5` se `x` è stata dichiarata alla riga 12.

Il problema è che il nostro `Address` manteneva solo il nome e il tipo, senza la posizione. La posizione esisteva già nell'albero tipizzato ma il generatore di TAC la scartava ogni volta che costruiva un indirizzo. La correzione che abbiamo fatto è stata quella di aggiungere il campo mancante ad `AddrVar` e poi seguire, in `TacGen.hs`, ogni punto dove nasceva un `AddrVar` per evitare di scartare quella posizione e facendola arrivare fino in fondo. Così facendo abbiamo risolto il problema che avevamo con le variabili usate come espressioni, gli elementi di array (anche quelli ricorsivi a più dimensioni), per gli indirizzi delle l-expression, per il lato sinistro degli assegnamenti e per le dichiarazioni con valore iniziale.

Il pretty-printer è contenuto nel file `PrintTac.hs`. Dalla soluzione di prima abbiamo che se la posizione di una variabile o di una funzione c'è il nome diventa `nome_riga`; altrimenti rimane il nome normale, senza aggiunte. Prima del codice il pretty-printer stampa anche una sezione dedicata ai dati statici, con l'elenco delle stringhe letterali usate nel programma e l'etichetta associata ad ognuna. Il resto del programma traduce ogni istruzione nello stesso modo che abbiamo spiegato nei capitoli precedenti.

Il file `Main.hs` è la funzione richiesta esplicitamente dal testo: dato un file, fa parsing, poi type-check, poi stampa il sorgente, e infine genera e stampa il TAC se non ci sono errori di tipo. Abbiamo deciso di tenere separati gli errori di parsing e la lista di errori di tipo in modo che fosse il messaggio fosse chiaro e ben distinguibile invece che un'unica lista di errori.

Abbiamo riadattato anche il makefile rispetto a quello originale generato da BNFC. Ora `make` compila a partire dai file già presenti. Il comando `make demo` esegue il programma su tutti i file di test nelle cartelle previste. Abbiamo organizzato i test in più cartelle:

- `tests/typecheck`: un caso di mutua ricorsione, un programma con cinque errori di tipo diversi tutti riportati insieme, un caso con il parametro per riferimento che deve essere una l-expression e un caso che controlla i puntatori nei parametri per riferimento
- `tests/errors`: i casi che il type checker deve rifiutare esplicitamente, cioè un confronto di uguaglianza fra due array, un confronto d'ordine fra due caratteri, un `break` usato fuori dal corpo di un ciclo e la variabile di un `for` modificata all'interno del proprio corpo
- `tests/tacgen`: un programma con `if`, un programma con array, `while`, puntatori e chiamata per riferimento insieme, e poi un test per ciascuno dei costrutti aggiunti in seguito, cioè l'indicizzazione tramite puntatore `(*p)[i]`, la copia degli array passati per valore e dell'assegnamento fra array, i letterali array con indicizzazione multidimensionale, l'assegnamento composto su una variabile scalare e su un elemento di array, ed infine il short-circuit fuori dalle guardie

Lanciando `make demo` su tutto l'insieme di test ogni file produce l'output atteso: i file con errori di sintassi segnalano l'errore e si fermano lì, i programmi con errori di tipo li elencano tutti insieme senza fermarsi al primo e tutti i programmi corretti arrivano fino al TAC finale con gli identificatori annotati nel formato `nome_riga` richiesto.